

UNIVERSITÉ DE GENÈVE

ANALYSE ET TRAITEMENT DE L'INFORMATION

14X026

TP A : Linear Algebra

Author: Donnet Michel Jean Joseph

E-mail: Donnet.Michel.Jean.Joseph@etu.unige.ch

September 2025



**UNIVERSITÉ
DE GENÈVE**

FACULTÉ DES SCIENCES
Département d'informatique

1 Theory questions

Use the slides and your notes to answer these questions. You do not need to write code. Provide short explanations and examples where possible.

1. What is a vector? What is a vector space? Give an example of a vector in $\mathbb{R}^2$ or $\mathbb{R}^3$.

A vector is a mathematical object that has a length, a direction and a sense.

A vector space is a space of vectors with linear operations such as addition between vectors and multiplication by a scalar. It can be generated by the combination of a few vectors, according to the dimension of the space. For instance, we can generate all vectors of a vector space of dimension 2 by using 2 vectors.

2. What do we mean by the “dimension” D of a dataset?

The dimension of a dataset indicates the minimum number of vectors useful to generate the entire dataset by combinations between these vectors.

3. Create two small datasets: one scalar dataset of 5 numbers, and one vector dataset of 5 vectors in $\mathbb{R}^3$. Compute their mean and variance. Then, write the formal mathematical definitions of mean and variance.

The dataset are the following:

$$A = \{1, 2, 3, 4, 5\} \in \mathbb{R}$$

$$B = \left\{ \begin{pmatrix} 3 & 9 & 6 \\ 8 & 4 & 5 \\ 5 & 6 & 6 \\ 0 & 8 & 5 \\ 6 & 3 & 2 \end{pmatrix} \right\} \in \mathbb{R}^3$$

- The mean is given by the formula 1

$$\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \text{ for } \mathbf{x}_i, \mu \in \mathbb{R}^d \quad (1)$$

So for the first dataset, we have the following computation:

$$\mu = \frac{1}{n} \sum_{i=1}^n A_i = \frac{1+2+3+4+5}{5} = 3 \in \mathbb{R}$$

And for the second dataset, we have the following computation:

$$\mu = \frac{1}{n} \sum_{i=1}^n B_i = \frac{1}{n} (\sum_{i=1}^n B_{i1} \quad \sum_{i=1}^n B_{i2} \quad \sum_{i=1}^n B_{i3}) = \frac{1}{5} (22 \quad 30 \quad 24) = (4, 4 \quad 6 \quad 4, 8) \in \mathbb{R}^3$$

- The variance σ^2 is given by the formula 2 by taking the square racin of the result

$$\sigma_j^2 = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2 \text{ for } \mathbf{x}_i, \sigma \in \mathbb{R}^d \quad (2)$$

So we have for the first dataset

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (A_i - \mu)^2 = \frac{1}{5} ((1-3)^2 + (2-3)^2 + (3-3)^2 + (4-3)^2 + (5-3)^2) = \frac{10}{5} = 2$$

And for the second dataset

– j = 1

$$\sigma_1^2 = \frac{1}{n} \sum_{i=1}^n (x_{i1} - \mu_1)^2 = \frac{1}{5} ((3-4.4)^2 + (8-4.4)^2 + (5-4.4)^2 + (0-4.4)^2 + (6-4.4)^2) = \frac{37.2}{5} = 7.44$$

– $j = 2$

$$\sigma_2^2 = \frac{1}{n} \sum_{i=1}^n (x_{i2} - \mu_2)^2 = \frac{1}{5} ((9-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (3-6)^2) = \frac{1}{5} (9+4+0+4+9) = \frac{26}{5} = 5.2$$

– $j = 3$

$$\sigma_3^2 = \frac{1}{n} \sum_{i=1}^n (x_{i3} - \mu_3)^2 = \frac{1}{5} ((6-4.8)^2 + (5-4.8)^2 + (6-4.8)^2 + (5-4.8)^2 + (2-4.8)^2) = \frac{10.8}{5} = 2.16$$

So

$$\sigma^2 = (7.44 \quad 5.2 \quad 2.16) \in \mathbb{R}^3$$

4. Define the scalar product. Compute it for two vectors from your vector dataset above. How does the scalar product allow us to compute the projection of one vector onto another?

The scalar product is defined for two vectors $\mathbf{x}$ and $\mathbf{y}$ like the formula 3

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^d x_i y_i = |\mathbf{x}| |\mathbf{y}| \cos(\theta) \quad \forall \mathbf{x}, \mathbf{y} \in S^d \quad (3)$$

with S the vector space of dimension d .

So for vector $\mathbf{x} = (3 \quad 9 \quad 6)$ and $\mathbf{y} = (8 \quad 4 \quad 5)$, the scalar product is given by the following calculs.

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^d x_i y_i = 3 \times 8 + 9 \times 4 + 6 \times 5 = 90$$

5. What is a basis of a vector space?

A basis of a vector space is a set of vectors that can generate the entire vector space by basics operations, such as addition between vectors and multiplication by scalars.

6. Explain the eigenvalue decomposition (EVD) and singular value decomposition (SVD). Why are they important in data science?

$X \in \mathbb{R}^{n \times n}$ is a square matrix. The eigen vector $\mathbf{u}$ is a vector that satisfies the equality 4.

$$(\exists \sigma \text{ such as } X\mathbf{u} = \sigma\mathbf{u} \text{ with } \mathbf{u} \neq 0) \Rightarrow (X - \sigma I)\mathbf{u} = 0 \quad (4)$$

So if X is diagonalisable, the matrix admits an Eigen Value Decomposition, given by the formula 5.

$$\forall i, X\mathbf{u}_i = \sigma_i \mathbf{u}_i \text{ and } \mathbf{u}_i \neq 0 \Rightarrow XU = U\Sigma \Leftrightarrow X = U\Sigma U^{-1} \quad (5)$$

Where

- $U \in \mathbb{R}^{n \times n}$ is a matrix composed of linearly independant eigen vectors $\mathbf{u}$
- $\Sigma \in \mathbb{R}^{n \times n}$ is a diagonal matrix with on the diagonal the values of the corresponding eigenvalues σ_i of the eigenvectors.
- U^{-1} the inverse of U .

Similarly, $\forall X \in \mathbb{R}^{n \times m}$, there exists a factorization $X = USV^T$ which is called the Singular Value Decomposition (SVD).

This two notions are very important in data science because they allow us to uncorrelate data, which is very useful for data analysis and processing because it saves a lot of time and ressources.

7. Give one concrete example of the curse of dimensionality, and one example of the blessing of dimensionality.
-

- Curse of dimensionality. A concrete example of this can be the data sparsity problem. We have an hypercube $[a, b]^D$ with D the dimensionality of the hypercube. We want to create an histogram that represents an empiric estimation of the probability density function (pdf) for this hypercube. For that, we quantize each dimension into b bins. Now, we hope an average of n samples per bin since if the number of samples per bin is too small, we can't have a good estimation because the density can vary a lot between bins. This generate noise for our estimation whereas with a great number of samples per bin, the variance will decrease and the empiric estimation will be closest to the real pdf, according to the Central Limit Theorem which says that when the number of samples increase, the empiric mean is closest to the real mean.
- Blessing of dimensionality. According to the lemma of Johnson-Lindenstrauss, we can find a linear projection that maps the datas in a space of smaller dimensionality that conserves the structure of our data with a distortion factor $\epsilon \in]0, 1[$. This lemma is a blessing of dimensionality because it says also that the projection depends only on the number of samples and not on the dimensionality of our data. So for instance, we have 10000 samples $\in \mathbb{R}^{3000}$ and we choose a distortion factor ϵ of 50% According to the lemma, we can project this datas with linear projection into a space of dimensionality given by the formula 6.

$$d = O\left(\frac{\log_2 N}{\epsilon^2}\right) = O\left(\frac{\log_2 10000}{0.5^2}\right) = O\left(\frac{13.28}{0.25}\right) \approx 50 \quad (6)$$

So we can reduce the dimensionality from 3000 to only 50 !!! This is an answer to the curse of dimensionality since it allows to work with big dimensionalities.

8. If you divide a D -dimensional space into b bins per dimension, and want k samples per bin, how many samples are needed in total?

If we divide each dimension into b bins, we have for D dimensions a number of bins:

$$\underbrace{b \times b \times \dots \times b}_{D \text{ times}} = b^D$$

Now, if we want k samples per bin, the total number of samples needed is:

$$N = k \times b^D$$

9. Why does data become more sparse as D grows? What happens if the number of samples stays fixed?

We suppose that we have a fixed number of samples N . The number of samples per bin is given by:

$$N = k \times b^D \Leftrightarrow k = \frac{N}{b^D}$$

So the expectation to have a sample x_i in a bin b_j is given by:

$$\mathbb{E}[P(x_i \in b_j)] = \frac{1}{N} \cdot \frac{N}{b^D} = \frac{1}{b^D}$$

When D increase, this expectation will approach 0, so it means that when D increase, most bins are likely to be empty since expectation to have a sample inside a bin approach 0.

10. Compare a hypersphere of radius r with its enclosing hypercube $[-r, r]^D$. As D increases, what happens to their volume ratio? What does this imply for uniform sampling?

We have the volum of the hypersphere of dimension D and radius r given by the formula 7.

$$\text{vol}(B^D(r)) = \frac{2r^D \pi^{D/2}}{D \Gamma(D/2)} \quad (7)$$

And the volum of the hypercube of $[-r, r]^D$ given by the formula 8.

$$\text{vol}(C^D(r)) = (2r)^D \quad (8)$$

So the ratio of this two volumes is given by the formula 9.

$$\frac{\text{vol}(B^D(r))}{\text{vol}(C^D(r))} = \frac{2r^D \pi^{D/2}}{D \Gamma(D/2)} \cdot \frac{1}{(2r)^D} = \frac{\pi^{D/2}}{D \Gamma(D/2) \cdot 2^{D-1}} \quad (9)$$

When D increase, we have that:

$$\lim_{D \rightarrow \infty} \frac{\text{vol}(B^D(r))}{\text{vol}(C^D(r))} = \lim_{D \rightarrow \infty} \frac{\pi^{D/2}}{D \Gamma(D/2) \cdot 2^{D-1}} = 0$$

So when the dimension is big, the volum of the hypersphere is neglectible compared to the volum of the hypercube. So this implies for uniform sampling that if we make uniform sampling in hypercube, there are no samples in hypersphere because the ratio of volumes is 0. It means that all samples will be concentrated, like the volum, in the corners of the hypercube and not in the hypersphere.

11. In high dimensions, what is the typical angle between two random vectors? How does this angle change as D increases?

We create two vectors by creating 4 random variables independant and identically distribued (iid) W, X, Y, Z in the space $\Omega \subset \mathbb{R}^D$ of dimension D . We define $u = X - Y$, and $v = Z - W$ two vectors. Now, we are interested to compute the $\mathbb{E}(\cos(\theta))$ between the two vectors. We have:

$$\mathbb{E}[\cos(\theta)] = \mathbb{E}\left[\frac{\langle u, v \rangle}{\|u\| \cdot \|v\|}\right] = \mathbb{E}\left[\frac{\langle X - Y, Z - W \rangle}{\|X - Y\| \cdot \|Z - W\|}\right]$$

As W, X, Y, Z are iid, $X - Y$ and $Z - W$ are also iid and centered. So $\mathbb{E}[X - Y] = \mathbb{E}[Z - W] = 0$ so we have $\mathbb{E}[\langle X - Y, Z - W \rangle] = 0$

As the denominator is a normalizer, $\mathbb{E}[\theta] = 0 \Rightarrow \theta = \frac{\pi}{2}$

So the two vectors seems to be quasi-orthogonal.

So when D increase, the vectors seems to be orthogonal.

12. What is meant by a “concentration phenomenon”? Give one example where it is useful, and one where it is problematic.

A concentration phenomenon is that in a high dimensional space, random variables tend to be concentrated around their mean value, with a variance that tend to be null, so with a probability of significant deviation that becomes small when D increase.

13. State one concentration inequality (e.g., Markov, Chebyshev, Hoeffding). Explain briefly what it tells us.

The Chebyshev inequality say that for a random variable X , a finite variance σ^2 and a finite expected value μ , it exists $\epsilon > 0$ such that:

$$\mathbb{P}(|X - \mu| > \epsilon) \leq \frac{\sigma^2}{\epsilon^2}$$

That means that when the variance decrease, the values are significant more close to the mean. It allows to estimate the contentration of data when we only know the variance of the datas, so it is very useful in case we analyse data without a knowledge of the distribution of the datas.

14. What is a PAC (Probably Approximately Correct) analysis in machine learning?

A PAC analysis in machine learning is when we try to train a model to learn a function, and we want a model approximately correct, that means with a small deviation or a small error. It means that we think that the function used to train the model is closest to the best function, based on a data sample. And we speak about probably because we want a strong probability to have a small error according to the data sample.

2 Matrix

1. Find a quadratic polynomial, say $f(x) = ax^2 + bx + c$, such that

$$f(1) = -5, \quad f(2) = 1, \quad f(3) = 11.$$

Write your solution here

2. Let a, b be some fixed parameters. Solve the system of linear equations

$$\begin{cases} x + ay = 2 \\ bx + 2y = 3 \end{cases}$$

$$\begin{cases} x + ay = 2 \\ bx + 2y = 3 \end{cases} \Rightarrow \begin{cases} x = 2 - ay \\ b(2 - ay) + 2y = 3 \end{cases}$$

$$\Rightarrow \begin{cases} x = 2 - ay \\ y(2 - ab) = 3 - 2b \end{cases}$$

$$\Rightarrow \begin{cases} x = 2 - a \frac{3 - 2b}{2 - ab} \\ y = \frac{3 - 2b}{2 - ab} \end{cases}$$

3. Find the inverse of the following matrix, if it exists:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}$$

The inverse of the matrix is given, according to pivot method:

$$\underbrace{\begin{pmatrix} 1 & 2 & 3 & | & 1 & 0 & 0 \\ 0 & 1 & 4 & | & 0 & 1 & 0 \\ 5 & 6 & 0 & | & 0 & 0 & 1 \end{pmatrix}}_{[A|I]} \Rightarrow \begin{pmatrix} 1 & 2 & 3 & | & 1 & 0 & 0 \\ 0 & 1 & 4 & | & 0 & 1 & 0 \\ 0 & -4 & -15 & | & -5 & 0 & 1 \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} 1 & 0 & -5 & | & 1 & -2 & 0 \\ 0 & 1 & 4 & | & 0 & 1 & 0 \\ 0 & 0 & 1 & | & -5 & 4 & 1 \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} 1 & 0 & 0 & | & -24 & 18 & 5 \\ 0 & 1 & 0 & | & 20 & -15 & -4 \\ 0 & 0 & 1 & | & -5 & 4 & 1 \end{pmatrix}$$

Hence

$$A^{-1} = \begin{pmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{pmatrix}.$$

4. Solve the following matrix equation for (X):

$$AX = B, \quad \text{where} \quad A = \begin{pmatrix} 1 & 3 \\ 2 & 5 \end{pmatrix}, \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \end{pmatrix}.$$

First, as $\det(A) = 1 \times 5 - 2 \times 3 \neq 0$, A is invertible. So we want to find A^{-1} with the same method than the precedent question:

$$\underbrace{\left(\begin{array}{cc|cc} 1 & 3 & 1 & 0 \\ 2 & 5 & 0 & 1 \end{array} \right)}_{[A|I]} \Rightarrow \left(\begin{array}{cc|cc} 1 & 3 & 1 & 0 \\ 0 & -1 & -2 & 1 \end{array} \right) \\ \Rightarrow \left(\begin{array}{cc|cc} 1 & 0 & -5 & 3 \\ 0 & 1 & 2 & -1 \end{array} \right)$$

So $A^{-1} = \begin{pmatrix} -5 & 3 \\ 2 & -1 \end{pmatrix}$. Now, we can solve the equation:

$$\begin{aligned} AX = B &\Rightarrow X = A^{-1}B \\ &\Rightarrow \begin{pmatrix} -5 & 3 \\ 2 & -1 \end{pmatrix} \times \begin{pmatrix} 7 & 8 \\ 9 & 10 \end{pmatrix} \\ &\Rightarrow \begin{pmatrix} 3 \times 9 - 5 \times 7 & 3 \times 10 - 5 \times 8 \\ 2 \times 7 - 1 \times 9 & 2 \times 8 - 1 \times 10 \end{pmatrix} \\ &\Rightarrow \begin{pmatrix} -8 & -10 \\ 5 & 6 \end{pmatrix} \end{aligned}$$

3 The importance of the mathematical concept behind a code

1. Download, open and run several times the PYTHON file “some_script.py”.

Explain the function `def project_on_first(u, v)` in geometrical terms and then explain the results of this code with clear mathematical concepts (you have to talk about projections and scalar product).

The execution of the script gives always “Result = 0.0”.

The function `def project_on_first(u, v)` is composed by a single code line:

```
1 return np.dot(u, v) / np.dot(u, u) * u
```

It correspond for the vector u and v to the scalar product between u and v divided by the scalar product between u and u (which give us a scalar) multiplied by u . If we write it in term of mathematic, we have: $\frac{\langle u, v \rangle}{\langle u, u \rangle} u$. This formula is the formula of the orthogonal projection of v on u .

In fact, in an orthogonal projection, we search a vector on u (that we call p) such as the vector $v - p$ is perpendicular to the vector u . As p is on u , we search a scalar λ to find our vector p : $p = \lambda u$.

So as the scalar product of 2 orthogonal vectors is 0, we have:

$$\langle v - \lambda u, u \rangle = 0 \Rightarrow \langle v, u \rangle - \lambda \langle u, u \rangle = 0 \Rightarrow \lambda = \frac{\langle v, u \rangle}{\langle u, u \rangle}$$

So the orthogonal projection is given by $p = \lambda u = \frac{\langle u, v \rangle}{\langle u, u \rangle} u$, which is what our function compute.

2. Consider $u = (u_i)_{1 \leq i \leq 7}$ and $v = (v_i)_{1 \leq i \leq 7}$, two vectors of length 7. How would you clearly write the following part of the script using some mathematical notation?

```
1 import numpy as np
2 u = np.random.randn(7)
3 v = np.random.randn(7)
4 r = 0
5 for ui, vi in zip(u, v):
6     r += ui * vi
```

Name the mathematics behind and rewrite the last 3 lines of code in one.

This code is simply the computation of the scalar product. In mathematical notation, we can write it like this: $\sum_{i=1}^n u_i \cdot v_i$ with $n = 7$ in our case.

We can replace the last 3 lines of code by the numpy function `np.dot` that compute the scalar product.

So the updated code is like this:

```
1 import numpy as np
2 u = np.random.randn(7)
3 v = np.random.randn(7)
4 np.dot(u, v)
```

3. Complete the code to construct a vector v orthogonal to the vector u and with the same norm as u . Comment each line of your code.

```
1 import numpy as np
2 u = np.random.randn(7)
3 v = np.random.randn(7)
4 # vector orthogonal to u obtained by the difference between v
5 # and a projection of v on u.
6 z = v - np.dot(u, v) / np.dot(u, u) * u
7 # normalize vector to have a norm of 1
8 z /= np.linalg.norm(z)
9 # multiply vector by norm of u to have same norm than u
10 z *= np.linalg.norm(u)
11 # print results
12 # As we work on discrete space, there is machine precision to consider.
13 print(f"Check orthogonality: {np.round(np.dot(z, u))}")
14 print(f"Norm of u: {np.linalg.norm(u)}")
15 print(f"Norm of created vector: {np.linalg.norm(z)}")
```

4. Given two vectors (u) and (v) in $\mathbb{R}^n$, write Python code to compute the cosine of the angle between them. Explain the mathematical concept behind this computation, and show how you would write it in one equation.

```
1 import numpy as np
2 u = np.random.randn(7)
3 v = np.random.randn(7)
4 # Get vector from orthogonal projection
5 p = np.dot(u, v) / np.dot(u, u) * u
6 # Get cosinus according to Thales theorem
7 cos_1 = np.linalg.norm(p) / np.linalg.norm(v)
8 # Same thing in one line because dot(..)/dot(..) is a scalar
9 cos_2 = np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))
10 # Print result in degrees
11 print(f"theta: {np.degrees(np.arccos(cos_1))}")
12 print(f"theta in multiple lines: {np.degrees(np.arccos(cos_2))}")
```

In the first way, we simply make an orthogonal projection, that create a rectangle triangle. Then we apply Thales theorem that indicate that $\cos(\theta) = \frac{\text{adjacent}}{\text{hypotenuse}}$ by taking the norm of our vectors (the norm used is euclidian norm that give us the euclidian distance from the vector).

In the second way, we use the fact that the scalar product can be written as $\langle u, v \rangle = \cos(\theta) \cdot \|u\| \cdot \|v\|$ (with $\|\cdot\|$ the euclidian norm). So it give us that $\cos(\theta) = \frac{\langle u, v \rangle}{\|u\| \cdot \|v\|}$. This formula is simpler to code in 1 line, and it's easy to proove equivalence between the two formulas obtained by developping and simplify the computation of $\frac{\|p\|}{\|v\|}$

4 Computing Eigenvalues, Eigenvectors, and Determinants

- Find the determinant, eigenvectors, and eigenvalues of the matrix

$$A = \begin{bmatrix} 5 & 6 & 3 \\ -1 & 0 & 1 \\ 1 & 2 & -1 \end{bmatrix}$$

Compare your results with those of the computer.

The determinant of a matrix can be computed by following the formula 10, according to the Laplace expansion.

$$\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{ij} \det(M_{ij}) \quad (10)$$

With M_{ij} the sumatrix obtained by removing the i -th row and the j -th column of A . So we have for $i = 1$:

$$\begin{aligned} \det(A) &= \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{1j}) \\ &= +5 \times (0 \times (-1) - (2 \times 1)) - 6 \times ((-1) \times (-1) - (1 \times 1)) + 3 \times ((-1) \times 2 - 1 \times 0) \\ &= -10 - 0 - 6 \\ &= -16 \end{aligned}$$

The eigen values are the result of the equation $\det(A - \lambda I) = 0$ with $\det(A - \lambda I)$ the characteristic polynomial of A .

$$\begin{aligned} \det(A - \lambda I) &= \begin{vmatrix} 5-\lambda & 6 & 3 \\ -1 & 0-\lambda & 1 \\ 1 & 2 & -1-\lambda \end{vmatrix} \\ &= (5-\lambda)(\lambda^2 + \lambda - 2) - 3\lambda - 6 \\ &= 5\lambda^2 + 5\lambda - 10 - \lambda^3 - \lambda^2 + 2\lambda - 3\lambda - 6 \\ &= -\lambda^3 + 4\lambda^2 + 4\lambda - 16 \end{aligned}$$

Now we search a first root of this polynom to factorize it. According to the rational root theorem, we search root that can divide 16.

- $\lambda = 1$:

$$-1^3 + 4 \cdot 1^2 + 4 \cdot 1 - 16 = -9$$

- $\lambda = 2$:

$$-2^3 + 4 \cdot 2^2 + 4 \cdot 2 - 16 = 0$$

So 2 is a root of the polynom. So we can factorize the polynom by $\lambda - 2$. To achieve our goal, we use x instead of λ to use a latex package that compute details of the operation.

$$\begin{array}{r|l} -x^3 + 4x^2 + 4x - 16 & x - 2 \\ \underline{x^3 - 2x^2} & -x^2 + 2x + 8 \\ 2x^2 + 4x & \\ \underline{-2x^2 + 4x} & \\ 8x - 16 & \\ \underline{-8x + 16} & \\ 0 & \end{array}$$

So we have the polynomial $(\lambda - 2)(-\lambda^2 + 2\lambda + 8) = (2 - \lambda)(\lambda^2 - 2\lambda - 8)$ As eigen values are given by the roots of the polynomial $\det(A - \lambda I)$, we have the following eigen values:

$$\begin{aligned}
 & (2 - \lambda)(\lambda^2 - 2\lambda - 8) = 0 \\
 \Leftrightarrow & \begin{cases} 2 - \lambda = 0 \\ \lambda^2 - 2\lambda - 8 = 0 \end{cases} \\
 \Leftrightarrow & \begin{cases} \lambda = 2 \\ \lambda = \frac{2 \pm \sqrt{(-2)^2 - 4 \cdot 1 \cdot (-8)}}{2 \cdot 1} \end{cases} \quad (\text{quadratic formula}) \\
 \Leftrightarrow & \begin{cases} \lambda_1 = 2 \\ \lambda = \frac{2 \pm \sqrt{36}}{2} = \frac{2 \pm 6}{2} \end{cases} \\
 \Leftrightarrow & \begin{cases} \lambda_1 = 2 \\ \lambda_2 = 4 \\ \lambda_3 = -2 \end{cases}
 \end{aligned}$$

Now, we will find a eigen vector $\mathbf{v}$ of the matrix A such that $(A - \lambda I)\mathbf{v} = \mathbf{0}$. We have:

- $\lambda_1 = 2$

$$A - \lambda I = \begin{bmatrix} 5-2 & 6 & 3 \\ -1 & 0-2 & 1 \\ 1 & 2 & -1-2 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 3 \\ -1 & -2 & 1 \\ 1 & 2 & -3 \end{bmatrix}$$

So we can establish the equation system as follow:

$$\begin{aligned}
 & (A - \lambda I)\mathbf{v} = \mathbf{0} \\
 \Leftrightarrow & \begin{cases} 3v_1 + 6v_2 + 3v_3 = 0 \\ -v_1 - 2v_2 + v_3 = 0 \\ v_1 + 2v_2 - 3v_3 = 0 \end{cases} \\
 \Leftrightarrow & \begin{cases} v_1 + 2v_2 = 0 \\ v_3 = 0 \end{cases} \\
 \Leftrightarrow & \begin{cases} v_1 = -2v_2 \\ v_3 = 0 \end{cases}
 \end{aligned}$$

So $v_1 = \begin{bmatrix} -2t \\ t \\ 0 \end{bmatrix}$ for $t \in \mathbb{R}$

- $\lambda_2 = 4$

$$A - \lambda I = \begin{bmatrix} 5-4 & 6 & 3 \\ -1 & 0-4 & 1 \\ 1 & 2 & -1-4 \end{bmatrix} = \begin{bmatrix} 1 & 6 & 3 \\ -1 & -4 & 1 \\ 1 & 2 & -5 \end{bmatrix}$$

So we can establish the equation system as follow:

$$\begin{aligned}
 & (A - \lambda I)\mathbf{v} = \mathbf{0} \\
 \Leftrightarrow & \begin{cases} v_1 + 6v_2 + 3v_3 = 0 \\ -v_1 - 4v_2 + v_3 = 0 \\ v_1 + 2v_2 - 5v_3 = 0 \end{cases} \\
 \Leftrightarrow & \begin{cases} v_1 = 9v_3 \\ v_2 = -2v_3 \end{cases}
 \end{aligned}$$

So $v_2 = \begin{bmatrix} 9t \\ -2t \\ t \end{bmatrix}$ for $t \in \mathbb{R}$

- $\lambda_3 = -2$

$$A - \lambda I = \begin{bmatrix} 5+2 & 6 & 3 \\ -1 & 0+2 & 1 \\ 1 & 2 & -1+2 \end{bmatrix} = \begin{bmatrix} 7 & 6 & 3 \\ -1 & 2 & 1 \\ 1 & 2 & 1 \end{bmatrix}$$

So we can establish the equation system as follow:

$$\begin{aligned} (A - \lambda I)\mathbf{v} &= \mathbf{0} \\ \Leftrightarrow \begin{cases} 7v_1 + 6v_2 + 3v_3 = 0 \\ -v_1 + 2v_2 + v_3 = 0 \\ v_1 + 2v_2 + v_3 = 0 \end{cases} \\ \Leftrightarrow \begin{cases} v_1 = 0 \\ v_3 = -2v_2 \end{cases} \end{aligned}$$

$$\text{So } v_3 = \begin{bmatrix} 0 \\ t \\ -2t \end{bmatrix} \text{ for } t \in \mathbb{R}$$

This is the code used to verify the results. Note that we use sympy to have exact values instead of numpy which is better for computation.

```

1  import sympy as sp
2
3  A = sp.Matrix([[5, 6, 3],
4                [-1, 0, 1],
5                [1, 2, -1]])
6  eigenvals = A.eigenvals()
7  eigenvects = A.eigenvects()
8  eigenvals = [i for i, _ in eigenvals.items()]
9  print(f"Eigen values: {eigenvals}")
10 print(f"Eigen vectors: {sp.pprint([(i, j) for i, _, j in eigenvects])}")

```

2. The covariance matrix for n samples $x_1, \dots, x_n$, each represented by a $d \times 1$ column vector, is given by

$$C = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T,$$

where C is a $d \times d$ matrix and $\mu = \sum_{i=1}^n x_i / n$ is the sample mean.

Prove that C is always positive semidefinite. (Note: A symmetric matrix C of size $d \times d$ is positive semidefinite if $v^T C v \geq 0$ for every $d \times 1$ vector v .)

A symmetric matrix C of size $d \times d$ is positive semidefinite if $v^T C v \geq 0 \forall v \in \mathbb{R}^d$.

We define X the matrix $[x_1 \ x_2 \ \cdots \ x_n]$ with x_i the column vectors, and M the matrix $\underbrace{[\mu \ \cdots \ \mu]}_{n \text{ times}}$ such as

$X, M \in \mathbb{R}^{d \times n}$ Then, we can write:

$$C = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T = \frac{1}{n-1} \underbrace{(X - M)}_{d \times n} \underbrace{(X - M)^T}_{n \times d}$$

Let $v \in \mathbb{R}^d$ a vector. We have:

$$v^T C v = v^T \frac{1}{n-1} (X - M)(X - M)^T v = \frac{1}{n-1} \underbrace{(v^T (X - M))}_{A \in \mathbb{R}^{1 \times n}} (v^T (X - M))^T = \frac{1}{n-1} A A^T$$

As $n \geq 1$, $\frac{1}{n-1} \geq 0$. As the dimension of A is $1 \times n$, $A A^T = \sum_{i=1}^n (A_i)^2$ and as a sum of squared number is always greater or equal to 0, we have that $\frac{1}{n-1} A A^T \geq 0 \iff v^T C v \geq 0$.

So the covariance matrix is always positive semidefinite.

3. In this portion of the exercise, we will calculate the eigenvalues of the covariance matrices of six data sets listed as follows:

filename	n	d	description
tp1_artificialdata[1-3]	1024	100	Artificial data generated from various auto-regression
tp1_artificialdata4	1024	100	Random Gaussian data
tp1_freyfaces	1965	560	Facial images of a man named Brendan
tp1_digit2	5958	784	Hand-written images of "2"

Download and unzip data for python.zip. Each file contains a $n \times d$ data matrix with rows representing n different samples in $\mathbb{R}^d$.

- Compute the covariance matrix of each dataset.
- Compute the eigenvalues for each covariance matrix.
- Compute the determinant of each covariance matrix.
- Compute the product of the eigenvalues for each covariance matrix.
- Display the eigenspectrum for each covariance matrix in a 2D plot.

Describe the relationship between the product of the eigenvalues and the determinant of each covariance matrix. In addition, describe the observations you made regarding the plot of the spectrum of eigenvalues of each covariance matrix. How can you explain the disparities between datasets?

We can see that the product of the eigenvalues is always 0, as the determinant for each covariance matrix. It's due to the relation between the product of the eigenvalues and the determinant. According to the eigen value decomposition, the covariance matrix (which is positive semidefinite) can be written as $A = U\Sigma U^{-1}$ with Σ the matrix composed by the eigen values of A .

So we have

$$\det(A) = \det(U\Sigma U^{-1}) = \det(U) \det(\Sigma) \det(U^{-1}) = \frac{\det(U)}{\det(U)} \det(\Sigma) = \det(\Sigma) = \prod_i \lambda_i$$

So the product of the eigenvalues is equal to the determinant of the covariance matrix. On top of that, we can see that the eigenspectrum has always the same form: a decrease curve that go around 0. We can constate that this curve decrease like inverse of exponential, and it's more marked when the number of data increase. As the eigen values represent the variance of the data in each dimension, it means that when the spectrum decrease quickly until values near 0, the variance of each dimension decrease also quickly. So the variance of a lot of dimensions is in the neighborhood of 0. It means that for this dataset, all the dimensions are not required because if the variance is near 0, all the data have approximatively the same value in this dimension. So looking at the spectre of the eigen value is very interesting to know the number of dimensions needed to represent our datas.

4. Let $A \in \mathbb{R}^{n \times n}$ be a square matrix. Show that if (A) has an eigenvalue $(\lambda = 0)$, then the matrix (A) is singular. Furthermore, prove that the determinant of (A) must be zero if any eigenvalue is zero.

Let $\lambda_1, \dots, \lambda_n$ the eigenvalues of A . The characteristic polynom p can be written as 11.

$$p(x) = \det(A - xI) = (\lambda_1 - x)(\lambda_2 - x) \cdots (\lambda_n - x) = \prod_i^n (\lambda_i - x) \quad (11)$$

If we evaluate this polynom in $x = 0$, we obtain the following result 12.

$$p(0) = \det(A - 0I) = \det(A) = \prod_i^n \lambda_i \quad (12)$$

So if A has an eigenvalue $\lambda_i = 0$, then we have

$$\prod_i^n \lambda_i = 0 = \det(A)$$

So $\det(A) = 0$ so the matrix is singular. So if any eigenvalue is 0, the determinant of the matrix is equal to 0 and the matrix is singular.

5 Computing Projection Onto a Line

There are given a line $\alpha : 3x + 4y = -6$ and a point A with the coordinates $(-1, 3)$.

1. Find a distance from the point A to the line α using linear algebra (not coordinate method).
2. Explain your code using a sketch and some mathematics (there should be a scalar product somewhere).

1. First, we define a point on the line to create vector. To achieve our goal, I choose $x = -2$. So I have

$$3x + 4y = -6 \iff 4y = -6 + 6 \iff y = 0$$

So the point $B = (-2, 0)$ is on the line α . We can define a vector $\vec{n} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ which is the vector normal, perpendicular to the line. We can define a vector from the point of the line to the point A : $\vec{u} = \begin{bmatrix} -1 + 2 \\ 3 - 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ Now, we want to find the orthogonal projection of the vector $\vec{u}$ on the vector $\vec{n}$. The orthogonal projection $\vec{p}$ give us:

$$\vec{p} = \frac{\langle \vec{u}, \vec{n} \rangle}{\langle \vec{n}, \vec{n} \rangle} \vec{n} = \frac{3 + 12}{9 + 16} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} \frac{9}{5} \\ \frac{12}{5} \end{bmatrix}$$

If we calculate the norm of this vector, we obtain the shortest distance between the point and the line. It give us:

$$\|\vec{n}\|_2 = \sqrt{\left(\frac{9}{5}\right)^2 + \left(\frac{12}{5}\right)^2} = \sqrt{\frac{81}{25} + \frac{144}{25}} = \sqrt{\frac{225}{25}} = \sqrt{9} = 3$$

So the distance from the point B to the line α is of 3.

2. My code is basically what I have explained in the precedent question.

```

1  import numpy as np
2
3  A = np.array([-1, 3])
4  B = np.array([-2, 0])
5  n = np.array([3, 4])
6  u = A - B
7
8  p = np.dot(n, u) / np.dot(n, n) * n
9  d = np.sqrt(sum(p**2))
10 print(f"Distance of the point to the line: {d}")

```